Using Self-Consistent Naive-Bayes to Detect Masquerades

Kwong H. Yung

Abstract. To gain access to account privileges, an intruder masquerades as the proper account user. This paper proposes a new strategy for detecting masquerades in a multiuser system. To detect masquerading sessions, one profile of command usage is built from the sessions of the proper user, and a second profile is built from the sessions of the remaining known users. The sequence of the commands in the sessions is reduced to a histogram of commands, and the naive-Bayes classifier is used to decide the identity of new incoming sessions. The standard naive-Bayes classifier is extended to take advantage of information from new unidentified sessions. On the basis of the current profiles, a newly presented session is first assigned a probability of being a masquerading session, and then the profiles are updated to reflect the new session. As prescribed by the expectation-maximization algorithm, this procedure is iterated until the probabilities and the profiles are consistent. Experiments on a standard artificial dataset demonstrate that this self-consistent naive-Bayes classifier beats the previous best-performing detector and reduces the missing-alarm rate by 40%.

Index Terms: naive-Bayes classifier, self-consistent update, expectation-maximization algorithm, semisupervised learning; masquerade detection, anomaly detection, intrusion detection.

I. Introduction

THIS paper presents a simple technique for identifying masquerading sessions in a multiuser system. Profiles of proper and intruding behavior are built on the sessions of known users. As new, unidentified sessions are presented, the profiles are adjusted to reflect the credibility of these new sessions. Based on the expectation-maximization algorithm, the proposed *self-consistent* naive-Bayes classifier markedly improves detection rates.

A. Motivation

Unauthorized users behave differently from proper users. To detect intruders, behaviors of proper users are recorded and deviant behaviors are flagged. Typically, each user account is assigned to a single proper user with a specified role. Hence, future sessions under that user account can be compared against the recorded profile of the proper user. This approach to detecting masquerades is called anomaly detection.

More generally, intrusion detection is amenable to two complementary approaches, misuse detection and anomaly detection. Misuse detection trains on examples of illicit behavior; anomaly detection trains on examples of proper behavior. Because examples of intrusions are generally rare and novel attacks cannot be anticipated, anomaly detection is typically more flexible. In the context of masquerade detection, anomaly detection is especially appropriate because each user account is created with an assigned proper user, whose behavior must not deviate far from the designated role.

B. Previous Approaches

The reference dataset used in this study has been studied by many researchers. Schonlau and his colleagues [8] presented the dataset in full detail and reviewed six techniques used to analyze the dataset. Maxion and Townsend [6] later achieved better results by using the naive-Bayes classifier with updating, which serves as the basis for comparison. Yung [10] markedly improved detection rates but relied on confirmed updates. This paper also substantially improves over [6] and achieves results competitive with [10].

Both Maxion and Townsend [6] and Schonlau and his colleagues [8] tried to correct for the nonstationarity of user behavior by updating the user profiles. In [8], detectors with updating had fewer false alarms and generally outperformed the versions without updating. In [6], the naive-Bayes classifier with updating performed uniformly better than the naive-Bayes classifier without updating, by a wide margin.

The results of [6] and [8] demonstrate clearly that updating the detector produced significant improvements by lowering the number of false alarms. As new sessions are presented to the detector, the detector must classify the new sessions and also update the profile with these new sessions. In both [6] and [8], the detector considers each new session separately in sequence and must decide immediately how to classify the new session. Once the detector decides that the new session is proper, the detector then updates the profile of the proper user to include the new session, as if the new session had been part of the training set.

C. Early Limitations

There are at least two limitations to the updating scheme used in [6] and [8]. Principally, the detector must make a concrete decision whether to update the proper profile with the new session. In many cases, the judgment of a session is not always clear. Yet the detector is forced to make a binary decision before the detector can continue, because future decisions depend on the decision for the current case.

Furthermore, all future decisions depend on the past decisions, but the converse does not hold. In principle, the detector can defer the decision on the current session until additional new sessions from a later stage are studied. Yet in both [6] and [8], the detector is forced to make an immediate, greedy decision

for each new session, without considering the information from future cases. Scores for previous sessions cannot be revised as new sessions are encountered. Hindsight cannot improve the detector's performance on earlier sessions, because backtracking is not allowed.

D. New Strategy

Without a concrete optimality criterion, there is no basis on which to judge the performance of the greedy sequential strategy. In general, however, the best greedy strategy is not always the best strategy overall. By ignoring strategies that involve backtracking, the greedy search may fail to make decisions most consistent with the entire dataset.

This paper presents an alternative strategy for identifying masquerading sessions. To start, an optimality criterion is defined by specifying a concrete objective function. Moreover, each new session is assigned a score indicating how likely that session is a masquerading session. Instead of a binary decision, the detector assigns to the new session a probability between 0 and 1. This new session is then used to update the detector in a nonbinary fashion. The technique presented in this paper is an extension of the naive Bayes classifier used in [6] and will be called the *self-consistent naive-Bayes* classifier.

E. Outline of Paper

Following this introduction, Section II provides a brief background on the standard naive-Bayes classifier used for detecting masquerading sessions. Section III then explains the EM-algorithm used in the self-consistent naive-Bayes classifier. Section IV presents results of experiments on a standard artificial dataset. Section V discusses the advantages of the self-consistent naive-Bayes classifier as well as potential for future work. Finally, Section VI summarizes this paper's main conclusions.

II. Background

The self-consistent naive-Bayes classifier is an extension of the usual naive-Bayes classifier. Basically, the EM-algorithm is used to assign probabilities to new unidentified sessions. By incorporating a new session in a nonbinary fashion, the strategy for updating the profile is no longer greedy but rather optimizes the log-likelihood function of an underlying probabilistic model. The details the probabilistic model are explained in Section III. There the formalism of the EM-algorithm is used to generate a procedure for maximizing the log-likelihood in the presence of incomplete data.

Although the self-consistent naive-Bayes classifier is a significant step beyond the simple naive-Bayes classifier used by Maxion and Townsend [6], three elements introduced there still apply here: the classification formulation, the bag-of-words model, and the naive-Bayes classifier. These three strategies are not unique to detecting masquerading sessions, but rather they appear in the far more general context of intrusion detection and text classification. Below, a brief review of the three elements appears in the context of identifying masquerading sessions.

A. Classification Formulation

As in much of intrusion detection, examples of normal behavior are plentiful, but intrusions themselves are rare. In the context of detecting masquerading sessions, many examples of proper sessions are available. In the training data, however, no masquerading sessions are known. Moreover, masquerading sessions in the test data are expected to be rare.

Naturally, proper sessions in the training data are used to build the profile of proper sessions. To create a profile for masquerading sessions, the sessions of *other* known users in the training data are used. For the dataset of [8], this approach is particularly appropriate, because the artificially created masquerading sessions do indeed come from known users excluded from the training set.

Each new session is compared against the profiles of the proper sessions and against the masquerading sessions. The anomaly-detection problem has been reformulated as a classification problem over the proper class and the artificially created intruder class. This reformulation allows many powerful techniques of classification theory to be used on the anomaly-detection problem.

Even in a more general context of anomaly detection, the profile of the proper sessions is not defined only by the proper sessions, but also by the sessions of other users. The extent of the proper profile in feature space is most naturally defined through both the proper cases and the intruder cases. So the classification formulation is potentially useful even for other anomaly-detection problems.

B. Bag-of-Words Model

The so-called bag-of-words model is perhaps the most widely used model for documents in information retrieval. A text document can be reduced to a bag of words, by ignoring the sequence information and only counting the number of occurrences of each word. For classification of text documents, this simple model often performs better than more complicated models involving sequence information.

Let $\mathbf{C} = \{1, 2, \ldots, C\}$ be the set of all possible commands. User session number s is simply a finite sequence $c_s = (c_{s1}, c_{s2}, \ldots, c_{sk}, \ldots, c_{sz_s})$ of commands. In other words, session number s of length z_s is identified with a sequence $c_s \in \mathbf{C}^{z_s}$. In the bag-of-words model, the sequence information is ignored. So c_s is reduced from the sequence (c_{sk}) into the multi-set $\{c_{sk}\}$.

As demonstrated in [6], the bag-of-words model outperforms many more complicated models attempting to take advantage of the sequence information. In particular, the techniques reviewed in [8] and earlier techniques based on a Markov model of command sequences achieved worse results.

C. Naive-Bayes Classifier

In the field of text classification, the simple naive-Bayes classifier is perhaps the most widely studied classifier, used in conjunction with the bag-of-words model for documents. The naive-Bayes classifier is the Bayes-rule classifier for the bag-of-words model, under the typical uniform loss function for misclassification.

Suppose that each user has a distinct pattern of command usage. Let c_s denote the sequence of commands in session number s.

By Bayes inversion formula, the posterior probability $P(u|c_s)$ of user u given the sequence c_s is

$$P(u|c_s) = \frac{P(c_s|u)P(u)}{P(c_s)} \propto P(c_s|u)P(u), \tag{1}$$

where $P(u)$ is the prior probability for user u, and $P(c_s|u)$ is the probability that the sequence c_s was generated by user u. In practice, c_s is assigned to the user $u_0 = \text{argmax}\ \{P(c_s|u)P(u) : u = 1, 2, \ldots, U\}$, among U different users. In other words, the session c_s is assigned to the user u_0 who most likely generated that session.

In the naive-Bayes model, each command is assumed to be chosen independently of the other commands. User u has probability p_{uc} of choosing command c. Because each command is chosen independently, the probability $P(c_s|u)$, that user u produced the sequence $c_s = (c_{s1}, c_{s2}, \ldots, c_{sk}, \ldots, c_{sz_s})$ of commands in session number s, is simply

$$P(c_s|u) = \prod_{k=1}^{z_s} P(c_{sk}|u) = \prod_{k=1}^{z_s} p_{uc_{sk}} = \prod_{c=1}^{C} p_{uc}^{n_{sc}}, \tag{2}$$

where $n_{sc} = \sum_{k=1}^{z_s} 1_{\{c_{sk}=c\}}$ is the total count of command c in session s.

III. Theory

The simple naive-Bayes classifier is typically built on a training set of labeled documents. Nigam and his colleagues [7] demonstrated that classification accuracy can be improved significantly by incorporating labeled as well as unlabeled documents in the training set. The algorithm for training this new naive-Bayes classifier can be derived from the formalism of the EM-algorithm.

This section continues the development of the probabilistic model introduced in Section II. First, the maximum-likelihood formalism is used to estimate the model parameters necessary for implementing the naive-Bayes classifier. Then the complete log-likelihood is introduced to incorporate new unidentified sessions. Finally, an iterative hill-climbing procedure for optimizing the log-likelihood is generated from the EM-algorithm.

A. Maximum-Likelihood Estimation

In Section II, the bag-of-words model for documents was introduced. The session was defined to be a sequence of commands chosen independently from the set of possible commands. Then the naive-Bayes classifier was derived from the simple Bayes inversion formula. In practice, the parameters of the probabilistic model must be estimated from the data itself.

From now on, only two distinct classes of sessions are considered, the proper class and the masquerading class. The indicator *variable* $1_s = 1$ exactly when session s is a masquerading session. Let $1 - \epsilon$ and ϵ be the prior probabilities that a session is proper and masquerading, respectively. Moreover, let p_c and p'_c be the probability of command c in a proper and masquerading session, respectively.

B. Likelihood from Test Sessions

The log-likelihood L_s of a test session s is simply, up to an additive constant

$$L_s = (1 - 1_s)(\log(1 - \epsilon) + \sum_{c=1}^{C} n_{sc} \log p_c) + 1_s(\log \epsilon + \sum_{c=1}^{C} n_{sc} \log p'_c) \tag{3}$$

where n_{sc} is the total count of command c in session s.

Assuming that all test sessions are generated independently of each other, the cumulative log-likelihood L^t after t test sessions is, up to an additive constant

$$L^t_+ = \sum_{s=1}^{t} L_s \tag{4}$$

$$= w^t_+ \log(1 - \epsilon) + \sum_{c=1}^{C} n^t_{+c} \log p_c + w'^t_+ \log \epsilon + \sum_{c=1}^{C} n'^t_{+c} \log p'_c \tag{5}$$

where

$$w^t_+ = \sum_{s=1}^{t} (1 - 1_s), \tag{6}$$

$$w'^t_+ = \sum_{s=1}^{t} 1_s, \tag{7}$$

$$n^t_{+c} = \sum_{s=1}^{t} (1 - 1_s) n_{sc}, \tag{8}$$

$$n'^t_{+c} = \sum_{s=1}^{t} 1_s n_{sc}. \tag{9}$$

Here w^t_+ and $w'^t_+ = t - w^t_+$ are the cumulative numbers of proper and masquerading sessions, respectively; n^t_{+c} and n'^t_{+c} are the cumulative counts of command c amongst proper sessions and masquerading sessions, respectively, in the t total observed test sessions.

C. Likelihood from Training Sessions

Now let n^0_{+c} denote the total count of command c among proper sessions in the training set. Likewise, let n'^0_{+c} denote the total count of command c among masquerading sessions in the training set. Letting r denote a session in the training set,

$$n^0_{+c} = \sum_r (1 - 1_r) n_{rc}, n'^0_{+c} = \sum_r 1_r n_{rc}. \tag{10}$$

Assuming that the sessions in the training set are generated independently, the log-likelihood L^0_+ of the proper and masquerading sessions in the training set is

$$L^0_+ = \sum_{c=1}^{C} n^0_{+c} \log p_c + \sum_{c=1}^{C} n'^0_{+c} \log p'_c. \tag{11}$$

This log-likelihood L^0_+ is useful for providing initial estimates of p_c and p'_c but provides no information about ϵ.

D. Posterior Likelihood

Rare classes and rare commands may not be properly reflected in the training set. To avoid zero estimates, smoothing is typically applied to the maximum-likelihood estimators. This smoothing can also be motivated by shrinkage estimation under Dirichlet priors on the parameters ϵ, p_c, and p'_c.

Here the parameters ϵ, p_c, and p'_c are drawn from known prior distributions with fixed parameters, and a simple standard Bayesian analysis is applied. Suppose that

$$(1-\epsilon,\epsilon) \sim \text{Beta}(\beta,\beta'), \quad (12)$$

$$p \sim \text{Dirichlet}(\alpha), \quad (13)$$

$$p' \sim \text{Dirichlet}(\alpha'), \quad (14)$$

where α, α', β, and β' are taken to be known fixed constants specified in advance. Then the cumulative posterior log-likelihood $\tilde{L}^t$ is, up to an additive constant

$$\begin{aligned}\tilde{L}^t &= (\beta-1+w_+^t)\log(1-\epsilon)+ \\ &\quad \sum_{c=1}^{C}(\alpha_c-1+n_{+c}^0+n_{+c}^t)\log p_c + \\ &\quad (\beta'-1+w_+'^t)\log\epsilon + \\ &\quad \sum_{c=1}^{C}(\alpha'_c-1+n_{+c}'^0+n_{+c}'^t)\log p'_c. \end{aligned} \quad (15)$$

Here w_+^t, $w_+'^t$, n_{+c}^t, and $n_{+c}'^t$, defined in Equations 6–9, are cumulative quantities determined by the t available test sessions; n_{+c}^0 and $n_{+c}'^0$, defined in Equation 10, are the fixed quantities determined by the training sessions.

E. Shrinkage Estimators

Equation 15 gives the cumulative posterior log-likelihood $\tilde{L}^t$ after observing t test sessions, in addition to the training sessions. Shrinkage estimators are just the maximum-likelihood estimators calculated from the posterior log-likelihood $\tilde{L}^t$. So cumulative shrinkage estimators $\hat{\epsilon}^t$, $\hat{p}_c^t$ and $\hat{p}_c'^t$ for ϵ, p_c and p'_c after $t>0$ sessions are

$$\hat{\epsilon}^t = \frac{\beta'-1+w_+'^t}{\beta-1+\beta'-1+t}, \quad (16)$$

$$\hat{p}_c^t = \frac{\alpha_c-1+n_{+c}^0+n_{+c}^t}{\sum_{v=1}^{C}(\alpha_v-1+n_{+v}^0+n_{+v}^t)}, \quad (17)$$

$$\hat{p}_c'^t = \frac{\alpha'_c-1+n_{+c}'^0+n_{+c}'^t}{\sum_{v=1}^{C}(\alpha'_v-1+n_{+v}'^0+n_{+v}'^t)}. \quad (18)$$

F. Complete Log-Likelihood

Initially, the training set is used to build the naive-Bayes classifier. As a new unidentified session is presented to the classifier, that new session is scored by the classifier and used to update the classifier. This scoring and updating procedure is repeated until convergence. As each new session is presented, the classifier is updated in a self-consistent manner.

For a session s in the training set, the identity is known. Specifically, $1_s=0$ for a proper session, and $1_s=1$ for a masquerading session in the training set. For a new unidentified session, 1_s is a missing variable that must be estimated from the available data.

G. EM-Algorithm for Naive-Bayes Model

The EM-algorithm [1] is an iterative procedure used to calculate the maximum-likelihood estimator from the complete log-likelihood. Each iteration of the EM-algorithm includes two steps, the expectation step and the maximization step.

In the expectation step, the indicators 1_s are replaced by their expectations, calculated from the current estimates of model parameters. In the maximization step, the new estimates of the model parameters are calculated by maximizing the expected log-likelihood. For each stage t, these two-steps are repeated through multiple iterations until convergence. Below the iterations of the EM-algorithm during a fixed stage t are described.

Let $\theta^t=(\epsilon^t,p_c^t,p_c'^t)$ denote the estimate of $\theta=(\epsilon,p_c,p'_c)$ at stage t, after observing the first t test sessions. For each known session r in the training set, 1_r is a known constant. So n_{+c}^0 and $n_{+c}'^0$ of Equation 10 remain fixed even as θ^t changes.

For each *unidentified* test session $s=1,2,\ldots,t$, the expectation step estimates $E[1_s|c_s;\theta^t]=P(1_s=1|c_s;\theta^t)$ via the Bayes inversion formula as

$$P(1_s=1|c_s;\theta^t)=\frac{P(1_s=1;\theta^t)P(c_s|1_s=1;\theta^t)}{P(c_s;\theta^t)}, \quad (19)$$

where

$$\begin{aligned}P(c_s;\theta^t) &= P(1_s=0;\theta^t)P(c_s|1_s=0;\theta^t)+ \\ &\quad P(1_s=1;\theta^t)P(c_s|1_s=1;\theta^t). \end{aligned} \quad (20)$$

For the expectation step of the current iteration, the estimate $\hat{\theta}^t$ from the maximization step of the previous iteration is used for θ^t.

The maximization step calculates updated estimate $\hat{\theta}^t$ of the parameter θ^t. The usual cumulative shrinkage estimators of Equations 16–18 are used. However, the counts w_+^t, $w_+'^t$, n_{+c}^t, and $n_{+c}'^t$ are now no longer known constants but rather are random variables. These random variables are replaced by their estimates from the expectation step above. In other words, the maximization step of the current iteration uses the estimates from the expectation step of the current iteration.

H. Sequential Processing of Sessions

Table I shows abstractly how the self-consistent update is used to process the incoming stream of test sessions. The algorithm processes in separate stages each new unidentified test session in sequence, from the first incoming session to the last incoming session. At stage t after sessions $s=1,2,\ldots,t$ have arrived, the EM-algorithm is run until convergence. In other words, at each stage t, multiple iterations of the expectation step and maximization step occur.

At the end of each *stage*, new consistent scores and models found after the EM-algorithm converges. These consistent scores and models are kept for each stage and used as the starting point for the next stage. For example, the opening iteration of the EM-algorithm for stage $t+1$ uses the closing values of the final iteration of the EM-algorithm in stage t. Estimates from the training set are used to initialize the very first iteration of the EM-algorithm at stage 1, after the first new test session arrives.

TABLE I

SELF-CONSISTENT UPDATE APPLIED TO SEQUENCE OF TEST SESSIONS.

- **Inputs:**
  - Target user u. Cut-off threshold h *not* necessary.
  - Initial training set $\mathbf{R}$ of identified sessions from all users.
  - Streamed sequence $\mathbf{S}$ of new unidentified test sessions for user u.
- Build initial classifier model $\hat{\theta}^0 = (\hat{\epsilon}^0, \hat{p}^0, \hat{p}'^0)$ for user u by using training set $\mathbf{R}$.
- Loop over each new session $t = 1, 2, \ldots, S$ in test sequence $\mathbf{S}$:
  - Start initial $\hat{\theta}^t = \hat{\theta}^{t-1}$.
  - Repeat until consistency:
    - E-step: Use Bayes inversion formula to calculate new scores $P(1_s = 1|c_s; \hat{\theta}^t)$, for $s = 1, 2, \ldots, t$, under current $\hat{\theta}^t$.
    - M-step: Update model $\hat{\theta}^t$, using new scores.
    - Keep new model $\hat{\theta}^t$ of stage t from current iteration for the next iteration.
  - Keep new scores for sessions $1, 2, \ldots, t$ under score set t.
- **Outputs:**
  - Classifier model $\hat{\theta}^S$ for target user u built on training set $\mathbf{R}$ and updated by the sequence $\mathbf{S}$ of sessions.
  - Multiple score sets, $t = 1, 2, \ldots, S$. Score set t contains scores of test sessions $s = 1, 2, \ldots, t$, updated based on information from all sessions $s = 1, 2, \ldots, t$ observed at stage t.

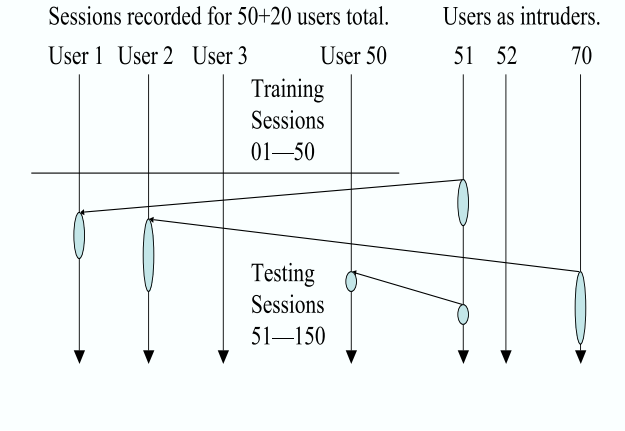

Fig. 1. Splicing of sessions.

I. Interpretation of Self-Consistency

From the computational perspective, the EM-algorithm allows the unidentified test sessions to be used. Since each new test session's identity is unknown, using that test session to update the profile blindly is risky. Instead, the EM-algorithm imputes a best guess on the identity of the unidentified session. This best guess acts as a placeholder and allows sequential classification to continue.

By imputing a probabilistic score for the identity of each unidentified test session, the EM-algorithm makes full use of the information in the unlabeled test sessions. In essence, the EM-algorithm improves masquerade detection by carefully incorporating the unlabeled test data in a meaningful way.

J. Multistep Paradigm and Backtracking

The multistep update is not as obvious as the one-step update but is indeed more intuitively appealing and logically sound. Because concept drift is unavoidable in masquerade detection, the detection rate depends not only on the underlying model but also the update scheme. The careful choice of update scheme is critical for the ongoing detection of masquerades. As a specific example of the multistep update, the self-consistent update based on the EM-algorithm has a firm statistical foundation and offers concrete optimality properties.

The backtracking in the multistep update naturally requires more computational effort. Additionally, because new scores are assigned to all previous sessions as each new session arrives in each successive stage, different modes of evaluation are possible.

IV. Results

The technique proposed in this paper is tested on a standard dataset previously analyzed by other authors using different techniques. The dataset contains command logs from 50 users on a UNIX multiuser system. First a brief overview of the reference dataset is provided. Then the experiments performed are described in detail. Finally, the results are compared with [6].

A. Reference Dataset

The dataset contains sequences of actual user commands, divided into artificial sessions of 100 commands each. For privacy reasons, the command logs were stripped of the command options and arguments, leaving only the commands themselves. Each original recording was declared to be free of intrusions. To create masquerading sessions, blocks of sessions from foreign users were spliced into the test set. This process of creating artificial masquerading sessions is illustrated in Figure 1. All intrusions appear only in discrete session units.

Originally command data was collected over a total of 70 users. From these 70 users, 20 users were selected at random to serve as intruders, as shown in Figure 1. Their sessions appear in the final dataset only as masquerading sessions for the 50 known users.

The first 50 sessions of the 50 users are declared to be free of intruders. These 50 sessions for the 50 users constitute the training set. The remaining 100 sessions for each of the 50 users form the test set. Sessions from the 20 excluded users were injected into the test portions of the 50 known users. The extra sessions in the test portions are then removed, leaving just 100 test sessions for each of the 50 known users.

To avoid artificially optimistic results, the following details of the data creation were *not* used as part of the classification procedures proposed in this report. Essentially, once an intruder has broken into a user account, he is likely to return on the very next session. In creating the reference dataset, a proper session is followed by a masquerading session with 0.01 probability. A masquerading session is followed by another masquerading session from the same intruder with 0.80 probability. A chain of masquerading sessions is taken from that intruder's log in a contiguous sequence. The exact details of this random splicing process are described in full by Schonlau and his colleagues [8].

B. Experimental Design

The details for one single experiment are described below. The example shows how masquerading sessions on User 12 were

detected. For other users, the same analysis applies.

The full training set was divided into two classes. The proper class consisted of 50 sessions for User 12. The masquerade class consisted of the 50 sessions for each of the other 49 users. So the masquerade class was built on a total of 2450 sessions.

The test set for user 12 had 100 sessions, all purportedly proper sessions. The classifier for user 12 was then run against each of the 100 sessions in a sequential fashion. As a new session is added, the scores of the previous sessions are also updated. At the end of the 100 sessions, the final set of all 100 scores were recorded.

The ROC curve for user 12 was generated by thresholding against the *final* set of scores, after all test sessions were presented sequentially. In this case, a session's score is the probability that the session was generated from the proper profile. Therefore, sessions with lower scores were flagged before sessions with higher scores.

C. Classifier Settings

A self-consistent naive-Bayes classifier was built for each individual user. The algorithm is outlined in Subsections III-G and III-H. The initial naive-Bayes was built on the training set. As each new session is presented, that new session is first scored by the previous classifier. Afterwards, all the unidentified sessions are used to construct the self-consistent naive-Bayes. This procedure is repeated for each new session.

In the full self-consistent naive-Bayes classifier presented in Section III, the fraction ϵ of masquerading sessions was adjusted in each iteration. In the results presented here, the fraction $\epsilon = 0.50$ was held fixed and was not estimated as part of the EM-algorithm. Keeping the prior weight $(1-\epsilon, \epsilon)$ fixed at $(0.50, 0.50)$ allowed for faster convergence and did not adversely affect the final results.

For each new session, the EM-algorithm was used to adjust the fractions p_c and p'_c. The Dirichlet parameters $\alpha_c = 1.01$ and $\alpha'_c = 1.01$ for all $c \in \mathbf{C}$ were chosen to match the parameters used in [6]. The EM-algorithm was iterated until the change between iteration i and $i+1$, measured by the quantity $||p^{(i+1)} - p^{(i)}||^2 + ||p'^{(i+1)} - p'^{(i)}||^2$ averaged over the total number of estimated parameters, was less than some tolerance, set to be 2.2×10^{-16}. In practice, the algorithm was not sensitive to the precise tolerance.

D. Composite ROC Curves

A separate classifier was created for each user and used to score that user's test sessions. The scores of test sessions from one user can be compared because those scores were assigned by that user's classifier. However, scores from different users cannot easily be compared because those scores are assigned from different classifiers.

To evaluate a strategy's success over the entire dataset of 50 users, a composite ROC curve can be useful. There are several common ways to integrate the individual 50 ROC curves into one single curve, but all methods are at best arbitrary.

In this paper, the scores from the 50 different classifiers were taken at face value. The 5000 total test sessions from the 50 user were sorted, and sessions with the lowest scores were flagged first. Because the scores were probability values, this global thresholding strategy has some merit. This method for constructing the composite ROC curve was also used in [6] and [8]. Because the same methodology was used in these previous papers, the composite ROC curves can be meaningfully compared.

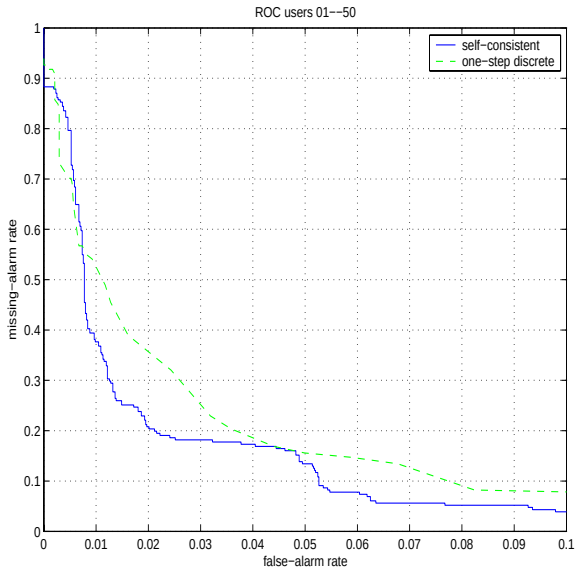

Fig. 2. ROC curve over users 01–50, less than 0.10 false-alarm rate.

E. Experimental Results

Two ROC curves are used to compare the self-consistent naive-Bayes classifier to the one-step discrete adaptive naive-Bayes classifier of [6]. These curves together demonstrate that the self-consistent naive-Bayes classifier offers significant improvements. All results reported here are based on offline evaluation, after all the test sessions have been presented sequentially. Online evaluation and deferral strategies are discussed in a separate paper.

Figure 2 shows in fine detail the composite ROC curves of all 50 users, for false-alarm rate 0.00–0.10. Indeed, the self-consistent naive-Bayes outperforms the adaptive naive-Bayes uniformly for all but the small portion of false-alarm rate 0.00–0.01. In particular, for false-alarm rate 0.013, the self-consistent naive-Bayes reduces the missing-alarm rate of the adaptive naive-Bayes by 40%.

Figure 3 shows the ROC curve for user 12 alone. As noted by the other authors [6], [8], user 12 is a challenging case because the masquerading sessions in the test set appear similar to the proper sessions. On user 12, the adaptive naive-Bayes performs worse than random guessing, but self-consistent naive-Bayes encounters little difficulty. In fact, the 50 ROC curves over each individual user show that self-consistent naive-Bayes typically outperforms adaptive naive-Bayes.

V. Discussion

The self-consistent naive-Bayes classifier outperforms the adaptive naive-Bayes classifier in all practical circumstances, where only low false-alarm rates can be tolerated. For false-alarm rate 0.013, the self-consistent naive-Bayes classifier lowers the missing-alarm rate by 40%.

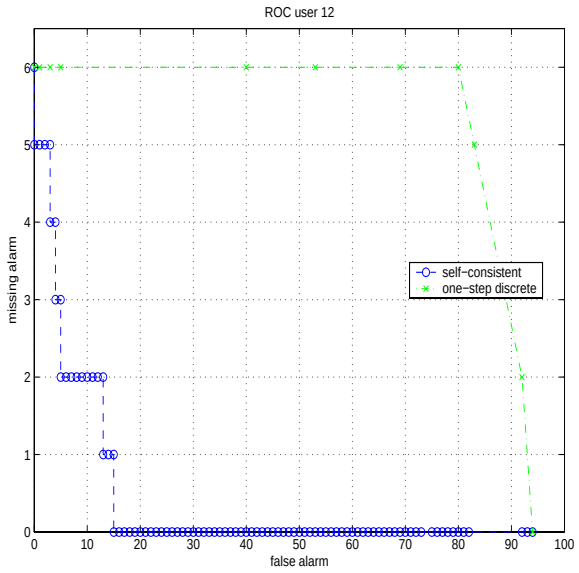

Fig. 3. ROC curve user 12

A. Parallel User Sessions

In a more realistic scenario, all 50 users would generate new sessions in parallel. In such a situation, changes to one user would affect other users directly. Because self-consistent naive-Bayes can change scores of earlier sessions as well as later sessions, the self-consistent naive-Bayes will benefit most from the additional information of other users. Although the adaptive naive-Bayes can use the additional information, only later sessions benefit because scores of earlier sessions cannot be changed.

B. Long-Term Scenario

Unlike the adaptive naive-Bayes, the self-consistent naive-Bayes is not forced to make a binary decision as each new session is presented. This advantage will become more dramatic as the number of users and the number of sessions increase. As the classifier learns from more cases, mistakes from earlier decisions are propagated onwards and magnified. Because the adaptive naive-Bayes is forced to make a binary decision immediately, it is also more likely to make errors. In a long-term scenario, the improvements of the self-consistent naive-Bayes classifier are expected to become far more pronounced.

C. Computation Time

Although the self-consistent naive-Bayes classifier offers a great number of advantages, more computation is required. The iterative procedure used to calculate the probabilities and to update the profile must be run until convergence. This additional effort allows the self-consistent naive-Bayes classifier to assign scores in a nongreedy fashion. Naturally, searching through the space of models requires more effort than the simple greedy approach.

In practice, convergence is often quick because most sessions have probability scores at the extremes, near 0 or 1. In the context of sequential presentation, only one single session is presented at a time. Computing the score of a single session requires little additional effort. Moreover, the set of scores from one stage can be used as the starting point to calculate scores for the next stage. This incremental updating approach relieves the potential computational burden.

D. Nonstationary User Behavior

For some users, the sessions in the test set differ significantly from the sessions in the training set. Because user behavior changes unpredictably with time, a detector based on the old behavior uncovered in the training set can raise false alarms. This high rate of false alarms can render any detector impractical, because there are not enough resources to investigate these cases.

In certain real world applications, however, identifying changed behavior may well be useful because a proper user can misuse his own account privileges for deviant and illicit purposes [5]. If the detector is asked to detect all large deviations in behavior, then flagging unusual sessions from the proper user is acceptable. This redefinition is especially useful to prevent a user from abusing his own privileges.

E. Future Directions

The iterative nature of the EM-algorithm is quite intuitive. In fact, many instantiations [2]–[4], [9] of the EM-algorithm existed well before the general EM-algorithm was formulated in [1]. Moreover, the self-consistency paradigm can be applied even to classifiers not based on a likelihood model. A self-consistent version can be constructed for potentially any classifier, in the same way that a classifier can be updated by including the tested instance as part of the modified training set.

The naive-Bayes classifier relies only on command frequencies. As a user's behavior changes over time, the profile built from past sessions become outdated. An exponential-weighing process can be applied to counts from past sessions. For the naive-Bayes classifier, this reweighting of counts is especially simple. Such an extension becomes even more valuable in realistic scenarios, in which the classifier is used in a sequential context over a long period of time.

VI. Summary

In previous approaches to detecting masquerading sessions, the detector was forced to make a binary decision about the identity of a new session. The self-consistent naive-Bayes does not make a binary decision but rather estimates the probability of a session being a masquerading session. Moreover, past decisions can be adjusted to accommodate newer sessions. Experiments prove that this sensible extension markedly improves over more restrictive adaptive approaches, by reducing the missing-alarm rate by 40%.

The self-consistent naive-Bayes classifier extends the usual naive-Bayes classifier by taking advantage of information from unlabeled instances. New instance are assigned probabilistic labels, and then the profiles are updated in accordance with the assigned probabilities of the new instances. This procedure is iterated until convergence to a final set of probabilities which are consistent with the updated profile.

By its very nature, the self-consistent naive-Bayes classifier is adaptive to the new sessions. Moreover, information from

new sessions is also used to adjust scores of previous new sessions. In this way, the scores of sessions are assigned in a self-consistent manner. As a specific instance of the EM-algorithm, the self-consistent naive-Bayes classifier finds a model optimal with respect to its likelihood given the available data.

VII. Acknowledgments

This research project was funded in part by the US Department of Justice grant 2000-DT-CX-K001. Jerome H. Friedman of the Stanford University Statistics Department provided invaluable advice through many helpful discussions. Jeffrey D. Ullman of the Stanford University Computer Science Department introduced the author to the field of intrusion detection and offered insightful critiques throughout the past three years. The author is grateful for their guidance and support.

References

[1] Arthur P. Dempster, Nam M. Laird, and Donald B. Rubin. "Maximum likelihood from incomplete data via the EM algorithm." *Journal of the Royal Statistical Society, Series B*, 39(1), 1–38, 1977.

[2] N. E. Day. "Estimating the components of a mixture of two normal distributions." *Biometrika*, **56**, 463–474, 1969.

[3] Victor Hasselblad. "Estimation of parameters for a mixture of normal distributions." *Technometrics*, **8**, 431–444, 1966.

[4] Victor Hasselblad. "Estimation of finite mixtures of distributions from the exponential family." *Journal of American Statistical Association*, **64**, 1459–1471, 1969.

[5] Vernon Loeb. "Spy case prompts computer search." *Washington Post*, 05 March 2001, page A01.

[6] Roy A. Maxion and Tahlia N. Townsend. "Masquerade Detection Using Truncated Command Lines." *International Conference on Dependable Systems and Networks (DSN-02)*, pp. 219–228, Washington, DC, 23–26 June 2002. IEEE Computer Society Press, Los Alamitos, California, 2002.

[7] Kamal Nigam, Andrew K. McCallum, Sebastian Thrun, Tom Mitchell. "Text classification from labeled and unlabeled documents using EM." *Machine Learning*, 39:2:103–134, 2000.

[8] Matthias Schonlau, William DuMouchel, Wen-Hua Ju, Alan F. Karr, Martin Theus, and Yehuda Vardi. "Computer intrusion: detecting masquerades." *Statistical Science*, 16(1):58–74, February 2001.

[9] John H. Wolfe. "Pattern clustering by multivariate mixture analysis." *Multivariate Behavioral Research*, **5**, 329–350, 1970.

[10] Kwong H. Yung. "Using feedback to improve masquerade detection." *Lecture Notes in Computer Science: Applied Cryptography and Network Security*. Proceedings of the ACNS 2003 Conference. LNCS **2846**, 48–62. Springer-Verlag, October 2003.